Summary of Agentic AI Systems (03/31)

Frank Fang (fangfr), Madeleine Hong (madehong), Evan Wang (evywang), Alan Zhang (alanzhng)

METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation

Problem and Motivation

With Retrieval Augmented Generation (RAG) systems, there is an inherent tradeoff between response quality and latency of response. While providing an LLM with more external context chunks generally improves accuracy, it significantly increases processing delays because long input contexts demand more compute and memory resources.

Traditional Retrieval Augmented Generation (RAG) systems are often underspecified, meaning the natural language queries do not dictate the exact execution parameters, such as how many text chunks to retrieve or how to synthesize them. Consequently, current systems typically rely on static, "one size fits all" configurations that are suboptimal. A simple query might retrieve too much data, wasting time, while a complex reasoning query might retrieve too little, sacrificing quality. Furthermore, previous research has treated configuration tuning and GPU scheduling (batching) as separate problems, failing to account for how a specific RAG configuration (like summarization) directly impacts GPU memory availability and scheduling efficiency.

METIS was built to address these challenges by optimizing the tradeoff between response quality and response latency.

Related Works

Prior research into optimizing RAG systems has generally followed two divergent paths: one focused on minimizing delay via query scheduling and one focused on maximizing quality via fine-tuning:

1. **Minimize delay:** utilizes scheduling-centric optimizations that aim to reduce response delay through efficient GPU allocation and inference batching. State of the art serving engines like vLLM and recent scheduling frameworks such as Parrot (OSDI 2024) and Sarathi-Serve fall into this category. While these systems excel at managing hardware resources, they typically treat the RAG configuration as a black box and do not adapt execution parameters, such as the number of retrieved chunks, on a per-query basis.
2. **Maximize quality:** focuses on quality-centric tuning, where researchers attempt to maximize the accuracy of RAG responses by refining the retrieval and synthesis stages. Frameworks such as AdaptiveRAG (ACL 2024) and RAG-studio explore how changing

configurations can improve performance on complex reasoning tasks. However, METIS identifies a critical gap in these approaches in that they often ignore the significant latency penalties incurred by higher quality configurations, such as the increased memory and compute requirements of joint synthesis methods. Furthermore, most existing systems rely on static configurations selected offline, failing to capture the per-query variability that METIS leverages through joint configuration adaptation and scheduling.

Solution Overview

METIS is the first RAG system designed to jointly schedule queries and adapt RAG configurations on a per-query basis to optimize the quality delay tradeoff. METIS employs a two-phase approach in order to handle the massive combinatorial space of possible configurations, which includes the number of chunks, the synthesis method (e.g., stuff, map rerank, or map reduce), and summary lengths.

Phase 1: Prune the configuration space with high accuracy as the focus

- METIS uses a separate, lightweight LLM profile to analyze incoming query alongside the database metadata and outputs four dimensions: Query complexity, joint reasoning, pieces of information, and summary length
- Using the information given from the lightweight LLM, METIS uses a rule-based mapping that determines which synthesis method to use

Phase 2: Joint optimizations with response delay and GPU memory usage as the focus

- Once the configuration space is pruned, METIS utilizes a resource-aware scheduler that calculates the GPU cost of every pruned configuration and selects the highest quality configuration for this set
- This ensures that the chosen configuration not only meets quality requirements but also fits optimally into available GPU memory to minimize overall response delay.

With this solution, METIS was able to achieve a lower response delay by 1.64-2.54x, higher throughput by 1.8-4.5x, and lower overall system cost by 2.38-6.8x.

Limitations

- The use of an LLM-based profiler introduces a nonzero computational overhead to every query. While the authors demonstrate this delay is relatively small, averaging only 3% to 6% of the total end-to-end delay, it still represents an additional step in the pipeline compared to static systems.
- The system currently relies on existing dataset summaries or metadata to guide the profiler; it does not yet include an automated mechanism to construct high-quality metadata for entirely new or unseen datasets.
- Current METIS relies on a rigid, rule-based mapping to translate LLM profile estimations into configurations, which is not suitable for all LLM tasks

- While the framework is designed to be extensible, the evaluation and primary mapping algorithms were specifically tailored for common RAG QA pipelines, meaning further adaptation may be required for more agentic or iterative RAG workflows.

Future Research Directions

1. Extending METIS framework to utilize the dynamic tuning beyond chunk count and synthesis methods to create agentic RAG workflows. This would introduce a number of sub-queries as a new optimization level, including selection of optimal embedding model, retrieval index, or even the underlying serving LLM.
2. Enhance underlying system efficiency by fine-tuning the confidence threshold. The authors of the paper suggest that the current 90% confidence threshold used to validate the profile outputs could be tuned more granularly to improve reliability across different domains.
3. Further explore how multi-turn chat history from the same user, as well as a community-based RAG could help profile and answer highly under-specified queries. For example, profiling the hierarchical summaries generated from knowledge graphs could be used to automatically select the optimal “community depth" for a given request.

Summary of Class Discussion

Q: In RAG queries, the chunk size is pre-determined. Do you think it would be a good idea to make the chunk size a tunable parameter?
A: No, because the profiling LLM understands the chunk size and the amount of information in each chunk, so there would be little benefit in changing the chunk size on each query. Also, to implement this, the chunks in the database need to be changed on every query, which isn’t realistic.

Q: How does the LLM profiler impact the latency and quality of response?
A: The paper says that the LLM profiler overhead accounts for about one-tenth of RAG execution delay, and from the results, we can see that the performance and throughput gain is greater than the loss. Although, there may be unpredictable performance behavior when the LLM has low confidence, and relies on the system fallback methods.

Q: Is either paper “implementable” for companies that have specific services in place for RAG?
A: HedraRAG has too many complexilities and tight couplings that hinder its ability to serve as a possible addition to corporate RAG services. However, METIS is flexible enough where it’s feasible that it is implemented. The knobs can be tuned to different parameters and in general, it touches fewer components than HedraRAG does.

HedraRAG: Co-Optimizing Generation and Retrieval for Heterogeneous RAG Workflows

Problem and Motivation

The primary motivation for HedraRAG is the increasing complexity and heterogeneity of modern Retrieval Augmented Generation (RAG) workflows, which now extend far beyond simple one shot retrieval and generation. Modern workflows often involve multihop reasoning, iterative retrieval, or speculative hypothesis generation, creating diverse request patterns that existing systems struggle to serve efficiently.

Existing work that have not adapted to the more complex RAG workflows run into the following problems:

1. Generation (which is typically GPU-bound) and retrieval (which is typically CPU-bound) are treated as separate stages. This causes pipeline bubbles, resulting in poor hardware utilization.
2. Accumulated latency in multi-round workflow due to dynamic batching makes it difficult to balance resources across concurrent requests
3. With large vector database, retrieval can be slow and cause a bottleneck

HedraRAG aims to adapt to the complex and heterogeneous RAG workflows by addressing these concerns of existing RAG systems.

Related Works

The landscape of RAG system development is currently dominated by modular frameworks and independent component optimizations. Popular open source frameworks like LangChain, LlamaIndex, Haystack, and FlashRAG provide developers with modular APIs to compose retrieval generation pipelines. While these tools are highly flexible, they typically dispatch retrieval and generation as separate, sequential stages. This stage-centric approach prevents them from exploiting more complex optimization opportunities, such as overlapping the execution of different stages across concurrent requests or reordering tasks based on semantic similarity.

On the hardware and library level, substantial work has been done to optimize individual RAG components. Vector search libraries like FAISS provide high performance CPU-based retrieval, while LLM serving engines like vLLM focus on maximizing GPU throughput for generation. However, HedraRAG notes that these components are often integrated into hybrid CPU-GPU designs without cross-stage co-optimization. Unlike prior efforts that focus on independently optimizing either the search or the model, HedraRAG moves beyond these boundaries by introducing the RAGraph abstraction. This allows the system to perform fine-grained transformations that were previously impossible in stage-centric frameworks, addressing the system-level inefficiencies inherent in serving heterogeneous, multistep RAG workflows.

Solution Overview

HedraRAG addresses the inefficiencies of modern RAG workflows by utilizing the following main ideas:

1. Parallelize independent generation and retrieval stages to improve throughput and minimize pipeline bubbles by introducing.
2. Utilize semantic similarity as many heterogeneous RAG workflows often have many rounds of generation and retrieval all with a similar context. Adjacent retrievals will likely get similar documents so we are able to cache them and partial generations are mostly accurate and can be used to pre-start the next retrieval.
3. Utilize inter-request retrieval skewness by using a GPU to store hot clusters
4. Use a graph based approach to customize and optimize easily for abstract heterogeneous RAG workflows.

HedraRAG is a codesigned system and made four major contributions:

1. RAGraph is a graph-based abstraction for RAG workflows. This representation allows the system to perform fine-grained transformations at runtime such as splitting large tasks into parallelizable sub-stages, reordering nodes to reduce dependencies, and rewiring the workflow to enable speculative execution.
2. Fine Grained Sub-Stage Pielining with a max time budget allows for the system to pre-add clusters from incoming retrieval requests while maintaining within budget
3. Similarity-Aware Search Optimizations allows for speculative generation and retrieval based on a partial search
4. Partial GPU Indexing leverages index skew by chasing hot clusters which reduces the latency associated with transferring data between the CPU and GPU.

Limitations

- The system must balance the latency gains of breaking stages into substages against the additional CPU cost of managing those substages. If the time budget for these substages is misconfigured, it can still lead to workload imbalances.
- Having a GPU index cache means less space for the KV Cache when resources are limited and thus, offline profiling is not sufficient
- Managing asynchronous cache updates, speculative execution rollbacks, and dynamic batching introduces significant overhead

Future Research Directions

1. RAGraph abstraction is designed to be extensible so future work could involve integrating a broader range of emerging workflow optimizations, such as automated prompt refinement or more advanced speculative decoding techniques.

2. Currently, determining KV Cache and hot cluster cache allocation is done with an offline profile. This instead could be done using a resource manager to adapt more at runtime for additional optimizations.
3. Further research could also explore tighter integration with various vector search algorithms beyond the ones evaluated (like FAISS), investigating how different indexing structures might change the optimal partitioning and scheduling logic of the hybrid CPU-GPU pipeline.

Summary of Class Discussion

Q: Is there any cost of mis-speculation? Does killing the speculation cost anything?
A: If the generation isn't good enough, it is still okay because the generation is happening in parallel with other requests. So in the worst case, the performance will operate at the same time as if it were to be served individually.

Q: How do they define if the partial search is equal to the full search (is partial good enough?)
A: If you have enough documents to do a partial generation, then you can evaluate the partial generation and if the retrieval was "good enough" to proceed. "Good enough" is determined by similarity scores between the speculated document retrieved and the full search document retrieved. Otherwise, the system will do a full retrieval.

Q: How is inter-retrival similarity optimized?
A: The main idea is that the cache will store documents that sequential queries might find useful. In theory, there could be lots of non-intersecting queries with low retrieval similarity. In a real world scenario, it's more likely that there won't be a completely random distribution of retrievals. For the same users, there's likely subsequent requests that have similarities, so caching the documents would be useful.

Q: Do you think this framework would work with tensor-parallelisms?
A: It could work, but there would need to be some cache synchronization, but this would cost more PCIe utilization, which is one of the resource bottlenecks that the paper seeks to avoid using. In general, people aren't running RAG systems on the same level as LLMs.